

CSCE 628 COMPUTATIONAL BIOLOGY · MODULE 02 · LECTURE NOTES

Sequence Alignment

Dynamic programming at its clearest.

MODULE OUTCOMES

- Derive the global alignment recurrence.
- Derive local alignment and say when to use it.
- Explain affine gap penalties and implement them.
- Explain the space and time trade-offs.
- Implement and verify an alignment engine.

1 Global alignment

$$F(i, j) = \max \{ F(i-1, j-1) + s(x_i, y_j), F(i-1, j) + g, F(i, j-1) + g \}$$

- **$F(i, j)$ is the best score for aligning the first i characters of x with the first j of y** — which is the subproblem definition, and **stating it precisely is most of the derivation** (CSCE 629 Module 05's discipline).
- **The first term aligns the two characters** — a match or a mismatch, **scored by the substitution function s** , which Module 03 §1 derives from observed data rather than choosing.
- **The second and third insert a gap in one sequence or the other** — **an insertion or a deletion, with penalty g** — and which of those it is depends on which sequence you call ancestral, which you usually cannot.
- **Three cases, one max, $O(mn)$ time and $O(mn)$ space with traceback** — and **the traceback pointer recovers the alignment itself rather than just the score**, which is the output anybody actually wants. **This is the recurrence to be able to write from memory**, because three later modules are variations on it.

AND EACH TERM CORRESPONDS TO AN EVOLUTIONARY EVENT, WHICH IS WHY THE MODEL IS CREDIBLE

A match or mismatch is a position that was conserved or substituted; a gap is an insertion or a deletion — so the alignment is a hypothesis about what happened between the two sequences and their common ancestor.

Which means the scoring parameters are a model of molecular evolution rather than arbitrary constants — and Module 03 §1 derives them from observed substitution frequencies in curated alignments, which is what makes the whole scheme more than a heuristic.

And the optimal alignment is the most likely history under that model — not the true history, which is unavailable and in most cases unknowable (Module 01 §4's no-ground-truth problem, in its first concrete instance).

So the thing to remember is: alignment is inference under a model — and the model's assumptions are exactly where the disagreements between different tools come from, which is why Project 1 asks you to align the same pair under two substitution matrices and report the difference.

2 Local alignment

SMITH-WATERMAN ADDS ZERO TO THE MAX, WHICH LETS AN ALIGNMENT START ANYWHERE

Add 0 as a fourth case in the max — so a negative running score resets to zero rather than persisting — and start the traceback from the maximum cell anywhere in the matrix rather than from the corner, ending when a zero is reached.

Which finds the best-matching *region* rather than the best whole-sequence correspondence — and that is almost always the biologically relevant question, because conservation is domain-wise rather than whole-sequence: two proteins may share one functional domain and nothing else.

So: global for sequences you believe are related along their entire length, local otherwise — and local is the default, including for database search (Module 04), where you have no reason to think the query resembles the whole of any database entry.

And the scoring must allow negative scores — a substitution matrix in which every entry is positive makes local alignment degenerate into global, since extending is never penalised — which is a real and easily missed implementation trap, and is why the expected score of a random pair must be negative (Module 03 §1).

3 Gaps

linear: $\text{cost}(k) = k \cdot g$ **affine:** $\text{cost}(k) = \text{open} + k \cdot \text{extend}$, with $|\text{open}| \gg |\text{extend}|$

- **The linear model is simple and biologically wrong: a single insertion or deletion event of length ten is far more likely than ten independent single-base events**, and the linear cost treats them identically.
- **Affine gives one expensive opening and cheap extension — which matches what is observed in curated alignments, and is what every real tool uses**, so an implementation without it will disagree with every reference.
- **The implementation needs three matrices: M (the last pair was aligned), I_x (a gap in x), and I_y (a gap in y) — one per state, each with its own recurrence over the three predecessors — which is Gotoh's algorithm, and it is still $O(mn)$.**
- **Three state matrices, still $O(mn)$ — and the affine recurrence is where hand implementations usually break** (Project 1's second requirement), typically by allowing a transition that opens a gap and immediately opens another.
- **The three-state formulation is the one to implement**, rather than the various two-matrix simplifications, because it is the one whose correctness is obvious by inspection.

4 Space

- **The score alone needs only two rows — $O(\min(m,n))$ space — because each cell depends only on the previous row and the current one**, which is the standard dynamic-programming space reduction (CSCE 629 Module 05).
- **But the traceback needs the whole matrix**, which is the quadratic memory cost — **and for two human chromosomes that is simply not available** on any machine.
- **Hirschberg's divide-and-conquer recovers the alignment in linear space — find where the optimal alignment crosses the middle column using two linear-space passes, then recurse on the two halves — at twice the time**, which is a very good trade.
- **And banded alignment restricts computation to a diagonal band** around the main diagonal — **correct only if the true optimal alignment stays inside the band, which is an assumption to state** rather than a safe optimisation, though it is usually fine for sequences of similar length.
- **Which is the pattern for the next three modules: exact alignment is quadratic, and everything faster assumes something** (Module 04's heuristics, Module 05's seeds) — **and the assumption is what should be reported.**

AND THE THING TO CARRY FORWARD

This recurrence is the most reused idea in the whole field — Modules 07 and 08 are the same dynamic programming on richer state spaces, and recognising that is most of the work of learning them.

Viterbi is alignment with probabilities instead of scores (Module 07 §2) — and the max-of-three structure is identical, with log-probabilities in place of substitution scores and transitions in place of gap penalties.

And RNA folding is the same idea on intervals rather than prefixes (Module 08 §2) — with a different decomposition and exactly the same discipline of stating the subproblem before writing the recurrence.

So implement this one carefully, because **three later modules are variations on it and the debugging habits transfer entirely** — particularly the habit of checking the base cases and the traceback separately from the fill.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Durbin et al., chapter 2 (library copy)	The whole module — alignment developed as probabilistic inference, which makes §1's callout precise.
Needleman & Wunsch (1970), and Smith & Waterman (1981)	§§1 and 2 in the originals — short, and the second is three pages.
Gotoh — An improved algorithm for matching biological sequences	§3's three-state formulation in the original.
Hirschberg — A linear space algorithm for computing maximal common subsequences	§4's divide and conquer — and it predates its biological application entirely.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Write the global recurrence from memory**, with base cases.
- 2 Fill a 5 by 5 matrix by hand** and trace back.
- 3 Implement Needleman-Wunsch** with traceback.
- 4 Implement Smith-Waterman** and find the best local region.

- 5 **Make every matrix entry positive** and observe local alignment degenerate.
- 6 **Implement affine gaps** with three state matrices.
- 7 **Compare linear and affine** on a pair with one long indel.
- 8 **Reduce the score computation to two rows.**
- 9 **Implement Hirschberg** and compare memory and time.
- 10 **Band the alignment** and find a case where the band makes it wrong.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 State the global recurrence and define the subproblem.
- 2 What does each of the three terms mean biologically?
- 3 What is the optimal alignment, exactly?
- 4 What single change gives local alignment, and what else changes?
- 5 When is local the right choice, and why is it the default?
- 6 Why must some matrix entries be negative?
- 7 Why is a linear gap cost wrong?
- 8 Describe the three-state affine formulation.
- 9 Give the space costs and Hirschberg's trade.
- 10 State the sentence that organises Modules 02 to 05.